Experiment 7

Aim: Implement Ensemble methods to combine different models.

Code:

```
# Step 1: Import library and load dataset
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# STEP 1
```

```
# Load the dataset
data = load_breast_cancer()

# Convert dataset into a DataFrame
df = pd.DataFrame(data.data, columns=data.feature_names)

# Add target column
df["target"] = data.target

# Display dataset information
print("First 5 rows:")
print(df.head(5))

print("\nDataset shape:", df.shape)

print("\nTarget distribution:")
print(df["target"].value_counts())

print("\nMissing values:", df.isnull().sum().sum())


# STEP 2

# Separate input features and target
X = df.drop("target", axis=1)
y = df["target"]

# Split data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Display dataset sizes
print("Original dataset:", df.shape)
print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target:", y_test.shape)



# STEP 3

# Create the Logistic Regression model
lr_model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000)
)

# Train the model
lr_model.fit(X_train, y_train)

# Predict on test data
```

```
y_pred = lr_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print("Logistic Regression Accuracy:", accuracy)
print("Accuracy Percentage:", round(accuracy * 100, 2), "%")



# Create the Decision Tree model
dt_model = DecisionTreeClassifier(random_state=42)

# Train the model
dt_model.fit(X_train, y_train)

# Predict on test data
y_pred_dt = dt_model.predict(X_test)

# Evaluate the model
dt_accuracy = accuracy_score(y_test, y_pred_dt)

print("Decision Tree Accuracy:", round(dt_accuracy * 100, 2), "%")



# Create the KNN model
knn_model = make_pipeline(
```

```
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)

# Train the model
knn_model.fit(X_train, y_train)

# Predict on test data
y_pred_knn = knn_model.predict(X_test)

# Evaluate the model
knn_accuracy = accuracy_score(y_test, y_pred_knn)

print("KNN Accuracy:", round(knn_accuracy * 100, 2), "%")

# STEP 4

individual_results = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Decision Tree",
        "KNN"
    ],
    "Accuracy (%)": [
        round(accuracy * 100, 2),
        round(dt_accuracy * 100, 2),
        round(knn_accuracy * 100, 2)
```

```
    ]
})

print(individual_results)

# STEP 6

# Create the Voting Classifier
voting_model = VotingClassifier(
    estimators=[
        ("lr", lr_model),
        ("dt", dt_model),
        ("knn", knn_model)
    ],
    voting="soft"
)

# Train the ensemble model
voting_model.fit(X_train, y_train)

# Predict on test data
y_pred_voting = voting_model.predict(X_test)

# Calculate accuracy
voting_accuracy = accuracy_score(y_test, y_pred_voting)

print("Voting Classifier Accuracy:",
```

```
      round(voting_accuracy * 100, 2), "%")

# STEP 7
# Create the Bagging Classifier
bagging_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)

# Train the model
bagging_model.fit(X_train, y_train)

# Predict on test data
y_pred_bagging = bagging_model.predict(X_test)

# Calculate accuracy
bagging_accuracy = accuracy_score(y_test, y_pred_bagging)

print("Bagging Classifier Accuracy:",
      round(bagging_accuracy * 100, 2), "%")

# individual_results.loc[len(individual_results)] = [
#     "Bagging Classifier",
#     round(bagging_accuracy * 100, 2)
# ]
```

```
# print(individual_results)

# STEP 8

# Create the Stacking Classifier
stacking_model = StackingClassifier(
    estimators=[
        ("lr", lr_model),
        ("dt", dt_model),
        ("knn", knn_model)
    ],
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5
)

# Train the model
stacking_model.fit(X_train, y_train)

# Predict on test data
y_pred_stacking = stacking_model.predict(X_test)

# Calculate accuracy
stacking_accuracy = accuracy_score(y_test, y_pred_stacking)

print("Stacking Classifier Accuracy:",
      round(stacking_accuracy * 100, 2), "%")
```

```
# individual_results.loc[len(individual_results)] = [
#     "Stacking Classifier",
#     round(stacking_accuracy * 100, 2)
# ]

# print(individual_results)

# STEP 9

# Store predictions from all six models
all_predictions = {
    "Logistic Regression": y_pred,
    "Decision Tree": y_pred_dt,
    "KNN": y_pred_knn,
    "Voting Classifier": y_pred_voting,
    "Bagging Classifier": y_pred_bagging,
    "Stacking Classifier": y_pred_stacking
}

# Calculate evaluation metrics
final_results = []

for name, predictions in all_predictions.items():
    final_results.append({
        "Model": name,
        "Accuracy (%)": round(
            accuracy_score(y_test, predictions) * 100, 2
```

```
        ),
        "Precision": round(
            precision_score(y_test, predictions), 4
        ),
        "Recall": round(
            recall_score(y_test, predictions), 4
        ),
        "F1 Score": round(
            f1_score(y_test, predictions), 4
        )
    })

results_df = pd.DataFrame(final_results)

# STEP 10

# Display the final comparison
print("FINAL MODEL COMPARISON")
print(results_df.to_string(index=False))

# Plot model accuracy
plt.figure(figsize=(10, 5))
plt.bar(results_df["Model"], results_df["Accuracy (%)"])
plt.title("Comparison of Individual and Ensemble Models")
plt.xlabel("Models")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)
```

```
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()

# Identify the highest test accuracy
best = results_df.loc[results_df["Accuracy (%)"].idxmax()]

print("\nHighest test accuracy:", best["Model"])
print("Accuracy:", best["Accuracy (%)"], "%")
```

Output:

```
(ml) PS C:\Users\Mukesh kushwaha\Documents\College\Codes\sem-7\ML> python exp7.py
First 5 rows:
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  mean compactness  ...  worst compactness  worst concavity  worst concave points  worst symmetry  worst fractal dimension  target
0        17.99         10.38          122.80     1001.0          0.11840           0.27760  ...             0.6656           0.7119                0.2654          0.4601                  0.11890       0
1        20.57         17.77          132.90     1326.0          0.08474           0.07864  ...             0.1866           0.2416                0.1860          0.2750                  0.08902       0
2        19.69         21.25          130.00     1203.0          0.10960           0.15990  ...             0.4245           0.4504                0.2430          0.3613                  0.08758       0
3        11.42         20.38           77.58      386.1          0.14250           0.28390  ...             0.8663           0.6869                0.2575          0.6638                  0.17300       0
4        20.29         14.34          135.10     1297.0          0.10030           0.13280  ...             0.2050           0.4000                0.1625          0.2364                  0.07678       0

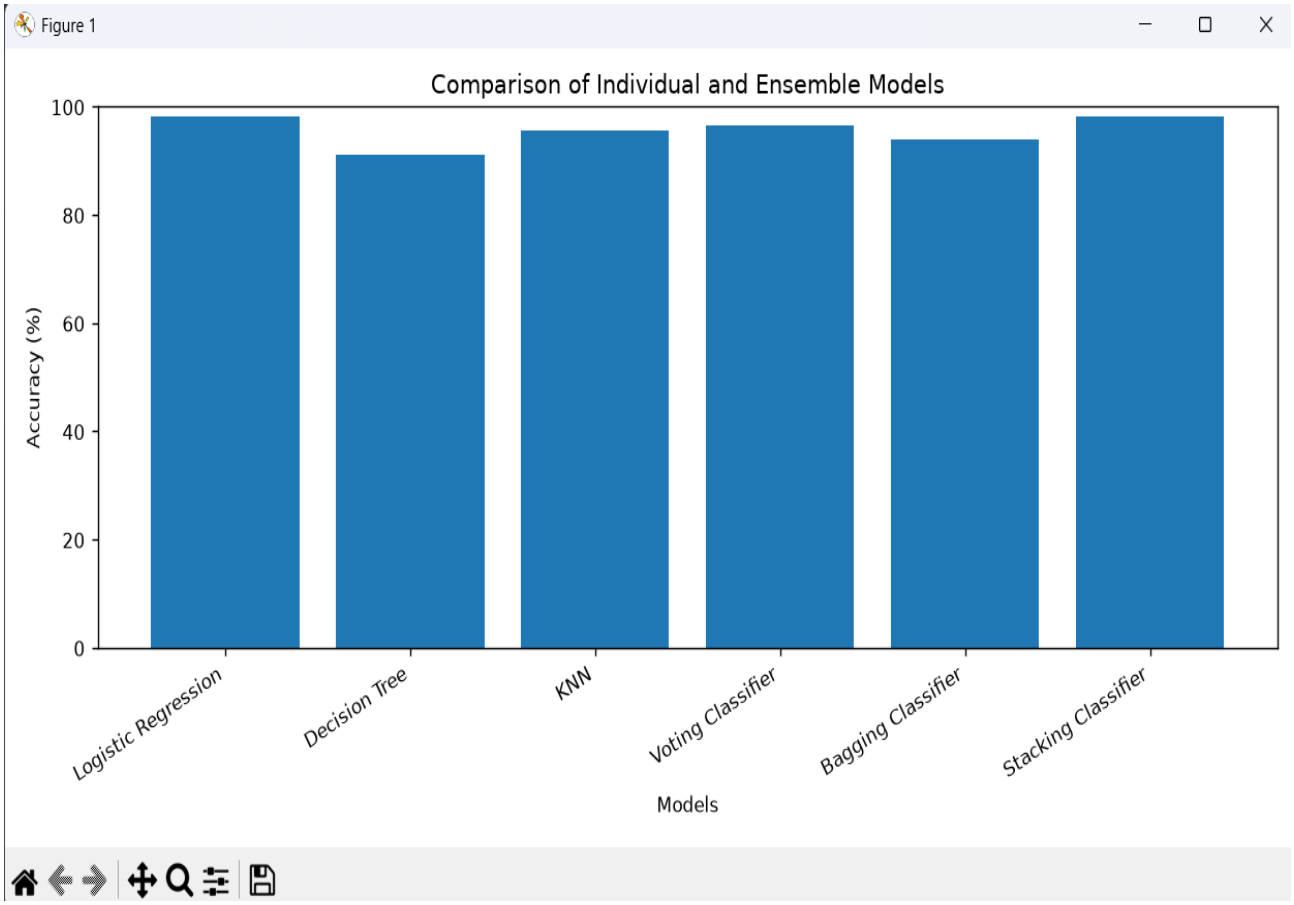
[5 rows x 31 columns]

Dataset shape: (569, 31)

Target distribution:
target
1    357
0    212
Name: count, dtype: int64

Missing values: 0
Original dataset: (569, 31)
Training features: (455, 30)
Testing features: (114, 30)
Training target: (455,)
Testing target: (114,)
Logistic Regression Accuracy: 0.9824561403508771
Accuracy Percentage: 98.25 %
Decision Tree Accuracy: 91.23 %
KNN Accuracy: 95.61 %
                 Model  Accuracy (%)
0  Logistic Regression         98.25
1        Decision Tree         91.23
2                  KNN         95.61
Voting Classifier Accuracy: 96.49 %
Bagging Classifier Accuracy: 93.86 %
Stacking Classifier Accuracy: 98.25 %
FINAL MODEL COMPARISON
                 Model  Accuracy (%)  Precision  Recall  F1 Score
Logistic Regression         98.25     0.9861  0.9861    0.9861
      Decision Tree         91.23     0.9559  0.9028    0.9286
                KNN         95.61     0.9589  0.9722    0.9655
  Voting Classifier         96.49     0.9595  0.9861    0.9726
 Bagging Classifier         93.86     0.9452  0.9583    0.9517
Stacking Classifier         98.25     0.9730  1.0000    0.9863

Highest test accuracy: Logistic Regression
Accuracy: 98.25 %
```


Figure 1
Comparison of Individual and Ensemble Models
Accuracy (%)
100
80
60
40
20
0
Logistic Regression
Decision Tree
KNN
Voting Classifier
Bagging Classifier
Stacking Classifier
Models